StreetCare AI — system architecture

Street-defect report review with deterministic screening, ML estimates, a multi-agent LLM workflow and a human decision gate

1 Presentation & access

Staff roles

three roles, sign-in required

- **Street Care Coordinator** — views queue · opens reviews
- **Highways Inspector (2 / district)** — decides reviews in own district
- **Customer Services Adviser** — status & chat only — no predictions

sign in

Dashboard (Streamlit)

dashboard/ — holds no business rule; every figure comes from the service

Sign in · Report queue · Report detail · Decision queue · Audit trail · Street Care chat

Pages shown per role (ROLE_PAGE_ACCESS) · api_client.call_service is the single HTTP client: base URL, timeout and session token

JSON +token

REST service (FastAPI)

api/routes.py · api/schemas.py — one sign-in & permission guard, one failure handler

may cost LLM calls

`POST /auth/signin` · `POST /auth/signout` · `GET /queue` · `GET /queue/{ref}` · `POST /reviews` · `GET /reviews` · `POST /reviews/{id}/decision` · `GET /reviews/{id}/calls` · `POST /chat/ask` · `GET /chat/turns` · `GET /audit/chat-turns`

access/: PBKDF2-HMAC salted password hashes + session tokens · role → permission → route (PERMISSION_BY_ROUTE), refused server-side · per-report locks

GET /queue · /queue/{ref} — *POST /reviews · decision (resume)* — */chat/ask*

2 Online reasoning & orchestration

Deterministic tool layer (tools/)

plain code shared by workflow, service and chat — never calls the LLM

- **report_data_tools.py** — live queue, documents, register; queue ranking
- **screening_tools.py** — Met / Not met / Not documented; street works; triggers
- **evidence_tools.py** — sentence typing; requirement links; disagreements
- **prediction_tools.py** — repeat likelihood + band; expected crew hours
- **retrieval_tools.py** — reviewer checks, guidance & policy sections
- **decision_tools.py** — conflict & refusal checks before resume
- **review_document_tools.py** — works-order / inspection facts; coordination brief

Agentic review workflow (LangGraph, agentic/)

LLM calls per run: 5 before gate · ORDER 7 · INSPECT 6 · REFER 5

10 nodes · each agent = exactly one LLM call that interprets tool-established facts and computes nothing

Load report context → **Planner** agent 1 → **Requirements** analyst · 2 → **Evidence** analyst · 3 → **Prediction** analyst · 4 → **Recommend.** writer · 5

review stored

Human-in-the-loop gate — await_inspector_decision

interrupt() · state check-pointed to SQLite (survives restart)
decider = Highways Inspector of the district who did NOT open it

ORDER · INSPECT — *REFER*

Order / inspection document drafter · 6 → *ORDER* → **Resident update** drafter · 7 (ORDER) → **Coordination brief** built by code (REFER) → **Record** outcome → DB

INSPECT

Street Care chat (chat/)

one question → at most one LLM call; grounded in documents only

- **conversation_memory.py** — asker's own last turns; record answers masked
- **street_care_assistant.py** — classify · retrieve passages · answer or refuse
- **record_answer.py** — report questions from review records, by role

Disclosure rules enforced in code, not by the model:

- street figures only when ≥ 5 reports make them up
- Customer Services never hears a model figure
- casebook excluded; ungrounded → fixed refusal
- every turn stored in chat_turns for audit

7 agent prompts — *1 prompt / question*

Language-model gateway — agentic/language_model.py

call_language_model · parse_labelled_response (fixed labelled fields) · key & model from .env · fixed temperature / max tokens
automatic function calling off · every accepted reply logged to llm_calls with tokens and latency

Google Gemini API

external LLM (google-genai)

writes via repository.py: runs · reviews · decisions · llm_calls · chat_turns · checkpoints

tools read

3 Knowledge, models & state

Knowledge base (ChromaDB)

16 PDFs → numbered sections → passages; a requirement is never split
embedded; exact search
section read-back for traceable citations

Defect-standard reader

knowledge/standard_reading.py

requirements + figures parsed from each type's own standard at run time — none in code
10 requirement kinds, exact decimals (min, max, footway / carriageway limits)

Live inputs (data/input/)

cleaned exactly like the training extracts

open_defect_reports.csv
45 rows → 40 open reports (latest rev.)
report_documents.csv
62 rows → 60 attached documents
street_works_register.csv
29 rows → 28 permits (23 live)

Trained models (output/models/)

scikit-learn pipelines + saved feature schemas

- **Repeat likelihood** — logistic regression · ROC AUC
- **Expected crew hours** — random-forest regressor · R², MAE
- **Sentence type (NLP, 9 classes)** — TF-IDF + logistic regression · macro-F1

Audit database (SQLite + SQLAlchemy)

persistence/repository.py — the only module that queries
user_accounts · agent_runs · report_reviews
review_decisions · llm_calls · chat_turns
checkpoint.db — LangGraph SqliteSaver
(paused runs; connection shared across threads)

4 Offline build (entry points)

index

run_knowledge_indexing.py

knowledge/

pdf_extraction
→ chunk_splitting (sections, passages, carried context)
→ vector_index (embed, persist, rebuild replaces store)
read-back check of every section

Raw sources (data/)

real data-quality defects

report_history_train/_eval
32,738 / 8,202 × 31 cols
repair_logs_train/_eval
26,095 / 6,476 × 27 cols
statement_sentences
36,661 × 8 cols
docs/ — 16 PDFs

run_data_pipeline.py

data_pipeline/

load as text; NULL_TOKENS → NaN; dedupe
canonical refs · category folding
unit conversion (exact decimal, half-up)
impossible values → missing; latest rev.
markup / entity / whitespace / CAPS fix
→ processed/*.csv + data_quality_report

fitted models

run_model_training.py

feature_builder.py · modeling/

+ requirement counts (screened under parsed standards) + trigger flags
fit on train; score on held-out eval
train vs held-out gap reported
saved only if all 3 meet minimums
→ models/ + model_metrics.json

tables + accounts

run_database_setup.py

persistence/ · access/

idempotent schema + seeded staff accounts (hashed)

run_api_service.py · run_dashboard.py

uvicorn + Streamlit

start-up checks that every earlier artefact exists

Human actor · User interface · Deterministic code · LLM-backed component · Trained ML model · Data / persistent store · Human decision gate

Arrows: data / control flow · dashed amber: prompt sent to the LLM.